

# TYPES OF RAG

## Naive RAG (Basic RAG)

Naive RAG is the simplest implementation of retrieval-augmented generation, where the system retrieves the top relevant documents from a vector database and directly feeds them into the language model without any filtering or optimization. It follows a straightforward pipeline: user query → embedding → retrieval → LLM response. However, since there is no ranking or refinement, irrelevant or noisy data may be included, which can reduce accuracy.

For example, a basic college FAQ chatbot that retrieves multiple documents about exam schedules and directly generates an answer from them is a typical use case of naive RAG.

## Advanced RAG

Advanced RAG enhances the basic RAG pipeline by improving the quality of retrieved data before sending it to the language model. It uses techniques such as query rewriting, hybrid search (combining keyword and semantic search), re-ranking of documents, and context filtering to ensure only the most relevant information is passed. This significantly improves accuracy and reduces hallucination.

For example, a legal AI assistant that refines the user query, retrieves multiple law documents, ranks them based on relevance, and selects only the most important sections to generate a precise answer is an example of advanced RAG.

## Modular RAG

Modular RAG structures the RAG system into independent components such as retriever, ranker, generator, and memory/cache, allowing each module to be developed, scaled, and optimized separately. This architecture is commonly used in large-scale production systems where flexibility and maintainability are important. Instead of a single pipeline, each module can be replaced or upgraded independently.

For example, an enterprise AI platform where the retrieval system runs as one service, the ranking model as another, and the LLM as a separate API is a real-world example of modular RAG.

## Hybrid RAG

Hybrid RAG combines multiple retrieval techniques, typically semantic search (vector-based) and keyword-based search (like BM25), to improve the relevance of retrieved documents. Semantic search helps understand the meaning of the query, while keyword search ensures exact matches are not missed. By combining both, the system achieves better recall and precision.

For example, an e-commerce search system where a user searches for “iPhone 13 battery issue” will benefit from both exact keyword matching (“battery issue”) and semantic understanding of the problem, making hybrid RAG very effective.

## **Graph RAG**

Graph RAG uses knowledge graphs instead of traditional document-based retrieval, where data is stored as entities (nodes) and relationships (edges). This allows the system to perform reasoning based on relationships between data points rather than just text similarity. It is particularly useful for complex queries involving connections between entities.

For example, in an organization, answering a query like “Who manages the employee working on Project X?” requires traversing relationships between employee, project, and manager, which is efficiently handled by graph RAG.

## **Agentic RAG**

Agentic RAG introduces intelligent agents that dynamically decide how to retrieve and process information instead of following a fixed pipeline. The agent can plan steps, refine queries, call external tools or APIs, and perform multi-step reasoning before generating the final answer. This makes it suitable for complex and multi-step queries.

For example, an AI research assistant that gathers data from multiple sources, compares results, and then generates a summarized explanation demonstrates agentic RAG.

## **Multimodal RAG**

Multimodal RAG extends traditional RAG by supporting multiple data types such as text, images, audio, and video. It converts all these formats into embeddings and retrieves relevant information across different modalities before generating a response using a multimodal language model. This enables richer understanding and more comprehensive answers.

For example, a medical AI system that analyzes an X-ray image along with a patient’s report and provides a diagnosis is a practical use case of multimodal RAG.

## **Adaptive RAG**

Adaptive RAG is a dynamic approach where the system decides whether retrieval is needed based on the user query. Instead of always performing retrieval, it uses a query classifier or decision mechanism to determine if the question can be answered directly by the LLM or requires external data. This reduces cost, latency, and unnecessary processing.

For example, if a user asks “What is machine learning?”, the system answers directly using the LLM, but if the question is “What is my company’s leave policy?”, it triggers retrieval to fetch internal documents, making adaptive RAG highly efficient.